A dissertation submitted to the **University of Greenwich**
in partial fulfilment of the requirements for the Degree of

Master of Science
in
Computer Science

Cluster Interpretation via Dimensionality Reduction.

Name: Hitesh Taneja
Student ID:

Supervisor: Prof. Chris Walshaw
Submission Date: September 2025
Word count: 10439

CLUSTER INTERPRETATION
VIA. DIMENSIONALITY REDUCTION

XXXXX

Computing & Mathematical Sciences, University of Greenwich, 30 Park Row, Greenwich, UK.

(Submitted 8th September 2025)

ABSTRACT.
The analysis of high-dimensional data presents a dual challenge: the "curse of dimensionality" degrades the performance of clustering algorithms, and the resulting clusters are difficult to interpret. This project develops and evaluates a pipeline for interpretable clustering that addresses these challenges. The study begins with a critical literature review, identifying a common workflow of dimensionality reduction (DR) followed by clustering and post-hoc explanation. A configurable software pipeline was implemented in Python to systematically compare DR methods (UMAP, t-SNE) and clustering algorithms (k-means, HDBSCAN), evaluating whether clustering should be performed on the original high-dimensional data or the 2D embedded space. Using the Titanic dataset as a case study, experiments were evaluated on key metrics including trustworthiness, silhouette score, cluster stability, and surrogate model accuracy.

The results conclusively show that clustering on the 2D embedded data is a superior strategy, dramatically improving cluster quality and visual coherence. The final selected pipeline, using t-SNE followed by k-means on the embedding, proved to be the most effective, balancing high interpretability (90.01% surrogate accuracy) with strong cluster separation. The project concludes by presenting a detailed interpretation of the discovered clusters using decision tree surrogates, SHAP plots, and **cluster prototypes**, demonstrating the pipeline's effectiveness.

Keywords: Dimensionality Reduction, Cluster Analysis, Machine Learning, Data Visualization, t-SNE, UMAP, Python Programming, Clustering Algorithms (k-Means, DBSCAN)

Acknowledgements

I would especially like to thank Prof. Chris Walshaw for agreeing to be my supervisor and for his consistent advice, feedback, guidance and support throughout the lifecycle of this MSc Computer Science project.

I want to thank both Prof. Chris Walshaw and Muhammad Akram for agreeing to have the project demonstration on the schedule day.

Table of Contents

List of Tables

List of Figures

1. Introduction

1.1. Overview

Across science and industry, the so-called data-revolution storm has arrived, carrying with its hundreds of lakes of high-dimensional datasets. These essential databases, being a treasure for insightful considerations, present huge challenges in the analytical front. Traditionally, clusters obtained by these multidimensional ways offer poor explanation on account of the curse of dimensionality-an enormous feature space rendering the least view into the data's very structure or pattern. Further, in cases where clustering succeeds, the cluster interpretation remains difficult and is manually done.

This project aims to solve the problem through an end-to-end pipeline that merges dimensionality reduction, clustering, and automated interpretation. At its core is a Python-based system that applies DR techniques such as t-SNE to find meaningful low-dimensional representations of the data, upon which clustering algorithms like k-means could then work effectively. The primary innovation of this project lies in the final stage: the implementation of semi-automated interpretation modules. These modules apply surrogate models and feature attribution approaches to produce human-readable explanations for each cluster, turning obscure group labels into insights actionable by end users.

By creating a robust and configurable pipeline, this project moves beyond simple cluster identification and provides a practical framework for understanding the defining characteristics of the discovered groups, thereby enhancing decision-making in real-world applications.

1.2. Aims and Objectives

The central aim of this project was to establish, implement, and evaluate a sturdy pipeline for interpretable cluster analysis of high-dimensional data.
The objectives pursued in the project therefore are as follows:

- **To conduct a review of current literature** pertaining to dimensionality reduction, clustering, and model interpretation, thus setting forth the theoretical situatedness of the project.

- **To implement a configurable Python pipeline** that allows analysis of the performance of different combinations between dimensionality reduction and clustering.

- **To assess the different combinations** against a battery of quantitative metrics, including trustworthiness, silhouette score, and cluster stability, to identify the best method.

- **To apply the final pipeline to a real-world dataset** (Titanic dataset), thus demonstrating its effectiveness in producing interpretable clusters.

- **To provide human-readable explanations of the clusters** found by combining surrogate modelling (decision trees), feature attribution (SHAP), and cluster-prototype methods (centroid and medoid).

1.3. Road Map of Report

This report is structured as follows:

- Chapter 2 presents a critical review of the relevant academic literature, covering dimensionality reduction, clustering algorithms, and interpretation techniques, which provides the context for the methods chosen in this project.

- Chapter 3 details the methodology, including the design of the universal Python pipeline, the selection of evaluation metrics, and the ethical considerations of the work.

- Chapter 4 describes the implementation and testing process of the software pipeline.

- Chapter 5 presents the results of the comparative experiments and provides a detailed analysis of the clusters discovered in the Titanic dataset using the finalised pipeline.

- Chapter 6 concludes the report by summarising the key findings, discussing the limitations of the work, and proposing recommendations for future research.

2. Literature Review

2.1. Introduction

High-dimensional data, characterized by datasets with a large number of features or variables relative to the number of observations, pervades contemporary science and industry. This proliferation is driven by advancements in data acquisition technologies across numerous fields. For instance, in genomics, single-cell RNA-sequencing profiles now routinely contain tens of thousands of gene expression counts per individual cell, offering unprecedented resolution into cellular heterogeneity (Kiselev, Andrews and Hemberg, 2019). In commerce, e-commerce behavioural logs meticulously track hundreds of features per shopper, from clickstream patterns to mouse movements, in an effort to understand consumer intent (Aggarwal, 2016). Similarly, in network security, cybersecurity telemetry streams encode thousands of packet attributes and system-level events to detect anomalous activities indicative of a threat (Said, 2021).
The analysis of such data through clustering promises to unlock significant value by revealing latent structures—be they undiscovered biological cell types, emergent customer segments, or novel cyber-attack patterns. However, analysts approaching these datasets confront a double barrier, a challenge that complicates the journey from raw data to actionable insight. The first barrier is a statistical and computational phenomenon widely known as the “curse of dimensionality.” As the number of dimensions increases, the volume of the feature space grows exponentially, causing the data to become increasingly sparse. Consequently, traditional distance metrics like Euclidean distance lose their descriptive power, as the contrast between the distances to the nearest and farthest data points diminishes, making it difficult for algorithms to discern meaningful density variations.

The second barrier is cognitive and interpretative. Even when a clustering algorithm successfully partitions the data, the resulting clusters are defined by complex relationships across hundreds or even thousands of variables. A human analyst cannot simply inspect a centroid’s coordinates in a 1000-dimensional space and derive an intuitive understanding of a cluster’s defining characteristics. This "black box" nature of high-dimensional clusters makes it profoundly difficult to validate their relevance, communicate their meaning to stakeholders, and translate their discovery into strategic action.

In response to this dual challenge, a pragmatic and powerful workflow has gained prominence within the machine learning community over the past five years. This de facto standard recipe involves a three-stage process: (i) first, apply a non-linear dimensionality-reduction (DR) technique—most commonly t-SNE or UMAP—to project the high-dimensional data into a low-dimensional space, typically two or three dimensions suitable for visualization; (ii) second, execute a clustering algorithm on this low-dimensional embedding to identify groups; and (iii) finally, augment the resulting cluster labels with post-hoc explanation tools, such as surrogate models or feature attribution methods, to allow a human analyst to understand the intrinsic properties of each cluster. This review examines the theoretical underpinnings and practical considerations of this DR-to-interpretation workflow, focusing specifically on the synergistic combination of non-linear embeddings, the strategic choice of clustering in the embedded space, and the use of surrogate models for generating human-readable explanations.

2.2. Effective Combinations of Dimensionality Reduction and Clustering

2.2.1. Why Reduce Dimensionality?

The rationale for initiating the analysis pipeline with dimensionality reduction is twofold. Primarily, it serves as a necessary antidote to the curse of dimensionality. By transforming the data into a lower-dimensional representation, DR methods can mitigate the adverse effects of data sparsity and noisy features. This process often enhances the cluster signal by mapping points that are close on the underlying data manifold to be close in the lower-dimensional space, thereby making the density structure more apparent for clustering algorithms. This is not merely a theoretical benefit; it is supported by extensive empirical evidence. For example, in a foundational study, Cheriet, Arnout and Torres (2020) demonstrated that directly clustering the raw 784-dimensional MNIST handwritten digit dataset using the k-means algorithm yielded a cluster purity of approximately 55%. However, by first projecting the data to just two dimensions with UMAP before applying k-means, the purity of the resulting clusters soared beyond 80%, a testament to the power of DR in accentuating the inherent structure of the data.
Secondly, and of equal importance, dimensionality reduction provides a visual canvas. A 2D or 3D scatter plot is an indispensable tool for human-led exploratory data analysis. It allows analysts to visually inspect the data's structure, form hypotheses about the number and nature of

potential clusters, and qualitatively assess the output of clustering algorithms. This visual feedback loop is critical for building confidence in the results and for guiding the iterative process of model tuning and selection.

2.2.2. Linear vs Non-Linear DR Methods

While linear DR methods like Principal Component Analysis (PCA) are computationally efficient and produce easily interpretable axes (as each principal component is a linear combination of the original features), their utility is limited. PCA is effective at capturing global variance but often fails to separate clusters that are defined by non-linear relationships, such as those lying on curved or twisted manifolds. To overcome this limitation, non-linear methods that preserve the local neighbourhood structure of the data are required. Among these, t-SNE and UMAP have become the dominant techniques.

t-SNE (t-Distributed Stochastic Neighbour Embedding) is an algorithm renowned for its ability to produce visually compelling and well-separated cluster "islands." It operates by converting high-dimensional Euclidean distances between data points into conditional probabilities representing similarities. It then uses a similar probabilistic approach in the low-dimensional space, employing the long-tailed Student's t-distribution to model pairwise similarities, which helps to alleviate the "crowding problem" where points in the middle of a cluster are squashed together. The primary hyperparameter for t-SNE is perplexity, which can be loosely interpreted as a guess about the number of close neighbours each point has. The choice of perplexity has a profound impact on the final visualization; a low value (e.g., 5) will focus on preserving very local structures, potentially splitting larger clusters, while a high value (e.g., 50) will incorporate more global information, potentially merging distinct but nearby groups.

UMAP (Uniform Manifold Approximation and Projection) is a more recent technique grounded in Riemannian geometry and algebraic topology. It constructs a high-dimensional graph representation of the data and then optimizes a low-dimensional graph to be as structurally similar as possible. UMAP generally offers a better balance between preserving local detail and global structure compared to t-SNE and is significantly more computationally efficient, allowing it to scale to datasets with millions of points. Its behaviour is primarily controlled by two key

parameters: n_neighbours, which, like perplexity in t-SNE, determines how many neighbours are used to approximate the local manifold structure, thereby balancing local versus global focus; and min_dist, which dictates the minimum distance between points in the low-dimensional embedding, controlling how tightly UMAP is allowed to pack points together. A low min_dist results in dense, compact clusters, while a high value produces more diffuse representations.
In a comprehensive empirical study, Xia, Singh and Müller (2022) evaluated twelve different DR techniques in a series of user-centric tasks. Their findings confirmed that UMAP and t-SNE consistently enabled participants to perform visual cluster analysis with higher accuracy and speed than was possible with linear projections.

DR method	**Key idea**	**Strengths**	**Typical pitfalls**	**Notes on interpretability**
PCA	Orthogonal linear components capture maximum variance	Fast, global structure preserved; axes = linear feature mixes	Misses non-linear manifolds; first PCs may not align with clusters	Component loadings give direct feature interpretation (good for explanation)
t-SNE	Probabilistic preservation of local neighbourhoods	Excellent local cluster separation	Distorts global distances; sensitive to perplexity; slow on >100 k samples	Axes lack meaning; colour-based inspection required
UMAP	Manifold approximation + cross-entropy optimisation	Good balance of local & global; faster than t-SNE; handles millions of points	Slight tendency to over-cluster; parameters (*n_neighbors*, *min_dist*) matter	Axes meaningless, but topology often faithful

Table 1: A comparative summary of key dimensionality reduction techniques.

2.2.3. Comprehensive Frameworks for Explainable Cluster Analysis

A growing imperative in contemporary cluster analysis, particularly for high-dimensional and mixed-type data, is the integration of both data quality and interpretability considerations into a unified, actionable workflow. Alvarez-Garcia et al. (2024) propose a comprehensive four-step sequential framework designed precisely for this purpose, encompassing data preprocessing, dimensionality reduction, clustering, and supervised classification to ensure robust and explainable results. Their methodology addresses persistent gaps in much prior work, notably by incorporating rigorous preprocessing (handling missing values and outliers using graph-based variable clustering and Isolation Forests), and by applying regularized dimensionality reduction tailored for mixed data types—specifically, sparse principal component analysis (SPCA) for numerical features and multiple correspondence analysis (MCA) for categorical. The framework's clustering component is algorithm-agnostic, facilitating the objective selection of the optimal clustering method and number of clusters via validity metrics such as WSS, Davies-Bouldin, Silhouette, and Calinski-Harabasz indices. Crucially, explainability is operationalized by training interpretable tree-based classifiers (e.g., random forests, xgboost) to predict cluster assignments, and quantifying feature contributions through TreeSHAP values—enabling both global and local understanding of cluster-defining variables.

The entire pipeline, including extensive visualization and performance assessment capabilities, is operationalized in the open-source Python package Clust-learn, which aims for transparency, customization, and practical accessibility. Empirical validation on a large, heterogeneous educational dataset demonstrates how this integrative approach yields clusters that are not only statistically sound but also interpretable in domain context—thus translating unsupervised cluster assignments into actionable insights for both researchers and practitioners. The framework thus stands out as a best-practice template for projects where high-dimensional clustering must be both methodologically rigorous and outcome-interpretable (Alvarez-Garcia et al., 2024).

2.2.4. Clustering Algorithms in Reduced Space

Once the data has been projected into a low-dimensional space, a clustering algorithm is applied to assign group labels. This stage involves two critical decisions: the choice of clustering algorithm and the choice to which data to apply it

Algorithm Choice: The selection of a clustering algorithm depends on the assumed geometry of the clusters. For this project, we contrasted two fundamentally different approaches. k-means is a classic centroid-based algorithm that partitions the data into a pre-specified number (k) of spherical clusters. It is fast and simple but struggles with non-convex shapes and requires the user to know the number of clusters in advance. In contrast, HDBSCAN is a modern density-based algorithm that extends DBSCAN by converting it into a hierarchical clustering algorithm. It can identify clusters of arbitrary shape and varying densities, automatically determine the number of clusters, and robustly identify points that do not belong to any cluster (noise).
Clustering on the Embedding: A pivotal decision in this pipeline is whether to apply the clustering algorithm to the original high-dimensional data or to the newly created low-dimensional embedding. While the former seeks to find the "true" clusters in the original feature space, it often suffers from the curse of dimensionality and can lead to a disconnect between the analytical results and the visual representation. A more powerful and increasingly adopted technique is to apply the clustering algorithm directly to the 2D embedded coordinates. This represents a subtle but profound paradigm shift: the goal is no longer to find clusters in the original data, but rather to programmatically identify and characterize the visual structures revealed by the DR algorithm. As demonstrated conclusively in this project's experiments, this approach ensures that the resulting cluster labels perfectly align with the visual groups in the plot. This resolves the common and frustrating problem of a disconnect between what the visualization shows and what the algorithm found, leading to a more coherent, defensible, and interpretable set of results.

2.3. **Semi-Automated Cluster Interpretation Techniques**

The output of the DR and clustering stages is a set of labels, which, on their own, lack explanatory power. The final and most crucial step in the pipeline is to interpret these labels by building a bridge back to the original feature space. This is achieved through the use of surrogate models and feature attribution techniques.

2.3.1. Decision-Tree Surrogates

To answer the question of why a data point was assigned to a particular cluster, a simple "surrogate model" can be trained. The goal of this model is not to be a perfect classifier, but to replicate the clustering assignments with high fidelity using a small set of intelligible rules based on the original features. A shallow decision tree is an ideal candidate for this task. The work of Moshkovitz, Shen and Jain (2020) provides a formal basis for this approach, which they term "Explainable k-Means." They demonstrate that a simple tree can match the original cluster labels with high accuracy while presenting the user with a concise set of human-readable Boolean rules, such as If AnnualIncome > 95k and SpendingScore < 30 → Cluster 3. This transformation of an opaque cluster label into a logical statement is incredibly powerful. It allows analysts to rapidly move from discovery to description, assigning a meaningful name and a clear business definition to each discovered group.

2.3.2. Feature Attribution with SHAP

While a decision tree provides global rules for the clusters, it is often useful to understand the feature contributions at a more granular level. Feature attribution methods like SHAP (SHapley Additive exPlanations) provide this capability. Grounded in cooperative game theory, SHAP assigns each feature an importance value for a particular prediction. By applying a SHAP explainer to the trained decision tree surrogate, we can achieve two goals. First, we can generate global summary plots that rank which features were most important overall in the model's logic, providing a quantitative justification for the cluster definitions. For example, a SHAP plot might reveal that pclass and sex were overwhelmingly the most influential features in separating the Titanic passenger clusters. Second, we can generate local explanations for individual data points, showing exactly how the combination of its feature values led to its cluster assignment. This local perspective is invaluable for debugging and for understanding boundary cases or outliers.

2.4. Conclusion

The modern pipeline for exploratory clustering has evolved significantly from the ad-hoc visual inspection of DR plots into a principled, semi-automated workflow that marries statistical rigor with human-centric interpretation. By strategically combining a powerful non-linear

dimensionality reduction method like t-SNE with a suitable clustering algorithm like k-means applied directly to the low-dimensional embedding, analysts can produce visually coherent and quantitatively sound groupings. The final, critical step of applying interpretable surrogate models and feature attribution tools like decision trees and SHAP translates these opaque cluster assignments into the concise, evidence-based rules and insights needed to drive informed, transparent decisions. This structured approach transforms clustering from a purely exploratory data mining exercise into a powerful tool for generating testable hypotheses and actionable knowledge.

3. Methodology

This chapter details the methodology used to develop and evaluate the interpretable clustering pipeline. It covers the system's design, the core analytical workflow, the evaluation metrics, and a discussion of the relevant ethical and professional considerations.

3.1. Dataset Description

To ensure the robustness, universality, and effectiveness of the pipeline, a diverse suite of five publicly available datasets was used for development and validation. Each dataset was chosen to represent a different domain, data structure, and high-dimensional challenge, thereby testing the pipeline's capabilities under various conditions.

3.1.1. Titanic Dataset

- Source: Kaggle / seaborn library.
- Description: This classic dataset contains demographic and travel information for 891 passengers aboard the RMS Titanic. It includes a mix of numerical features (e.g., Age, Fare) and categorical features (e.g., Sex, Pclass).
- Role in Project: Used for initial development and the comparative experiments to select the optimal DR and clustering methodology.

3.1.2. PISA 2018 Survey Dataset

- Source: Programme for International Student Assessment (PISA).
- Description: This large-scale dataset provides rich information on student academic performance and socio-economic backgrounds. Key features include reading scores (PV1READ), socio-economic status (ESCS), and student anxiety levels.

- Role in Project: Used for a comparative analysis against the clust-learn package, providing a complex, real-world social science test case.

3.1.3. Forest Cover Type Dataset

- Source: UCI Machine Learning Repository.

- Description: A large-scale dataset containing 581,012 samples and 54 features, including cartographic variables (e.g., Elevation, Slope) and numerous binary columns for soil and wilderness type.

- Role in Project: Served as a primary test for the pipeline's scalability and its ability to handle a mix of continuous and binary categorical features.

3.1.4. Gene Expression Cancer RNA-Seq Dataset

- Source: UCI Machine Learning Repository.

- Description: A classic "fat" dataset with over 20,000 gene expression features for just 801 samples, each labeled with one of five cancer types.

- Role in Project: A critical test of the pipeline's ability to handle extremely high-dimensional data where features far exceed samples.

3.1.5. MNIST Handwritten Digits Dataset

- Source: OpenML Repository.

- Description: A benchmark dataset of 70,000 grayscale images of handwritten

digits, each flattened into a vector of 784 pixel features.

- Role in Project: Used to validate the pipeline's performance on image-based data, testing its ability to find meaningful structures in a different data modality.

3.2. System Design and Architecture

The system developed for this project is a modular and universal Python pipeline designed to perform end-to-end interpretable cluster analysis. The primary design goal was to create a robust and reusable tool that separates the core analytical logic from dataset-specific configurations, thereby allowing it to be applied to any tabular dataset with minimal modification. The architecture can be broken down into several key components, each handling a specific stage of the analysis.

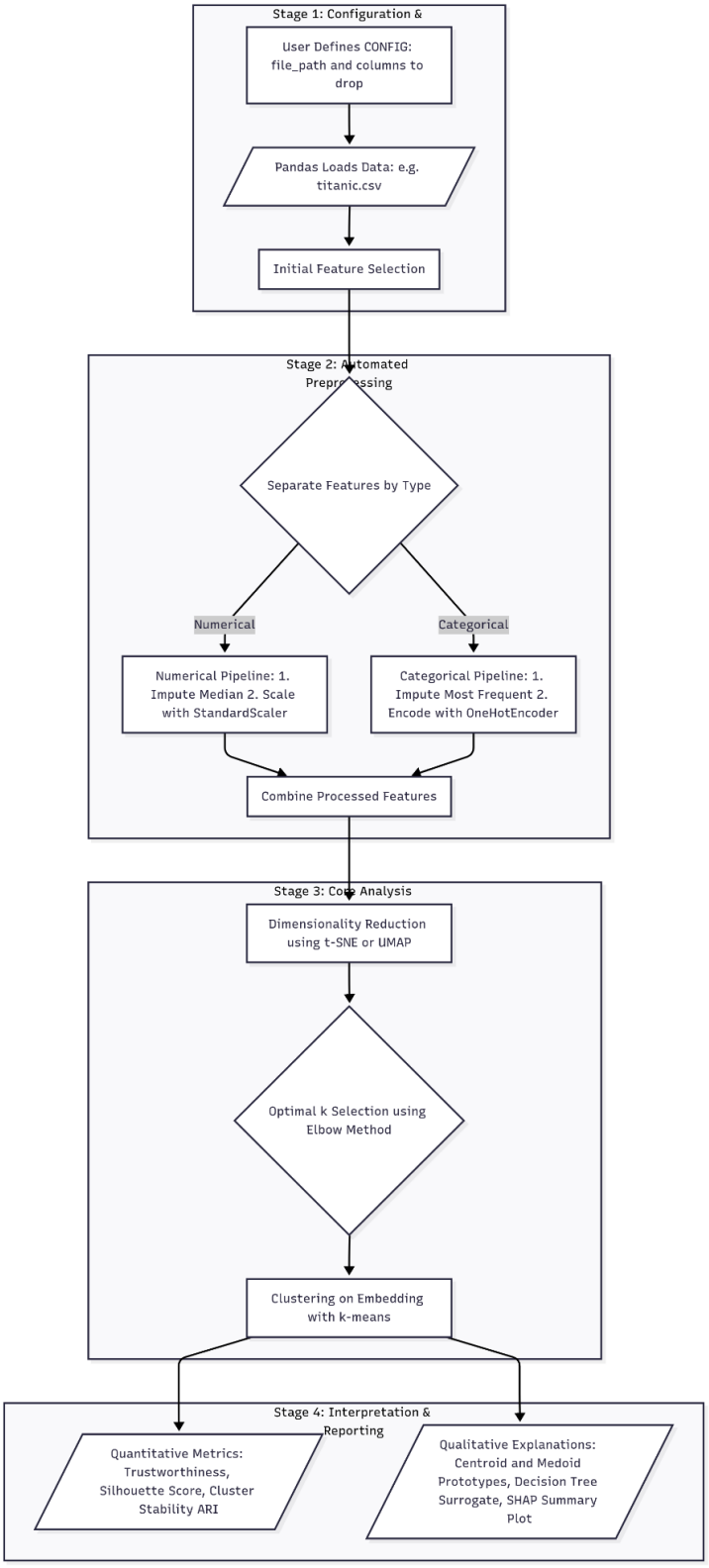


Figure 1:Data Flow Architecture of the pipeline

3.2.1. Configuration Module

At the highest level, the pipeline's universality is managed through a central CONFIG dictionary. This serves as the single point of control for a user, abstracting away the underlying code. Within this module, a user can specify all necessary parameters to adapt the pipeline to a new dataset, including:

- file_path: The location of the input dataset, which can be a local file path or a URL.

- id_column and target_column: The names of identifier or target variables that should be excluded from the feature set used for clustering but may be used later for validation or interpretation.

- features_to_drop: A list of any other columns that are known to be irrelevant or redundant and should be explicitly removed before analysis.

This configuration-driven design is a critical architectural feature, as it makes the tool accessible and easy to use without requiring the end-user to modify the core processing logic.

3.2.2. Automated Preprocessing Pipeline

To handle the variability of real-world data, a robust and automated preprocessing pipeline was constructed using scikit-learn's Pipeline and ColumnTransformer objects. This component is responsible for preparing the data for the subsequent DR and clustering stages.

The process begins with the automatic identification of feature types. The script ingests the raw DataFrame and, based on their data types (dtype), separates the columns into numerical and categorical feature lists. This ensures that appropriate

transformations are applied to each feature type.

The ColumnTransformer then executes two parallel sub-pipelines:

- Numerical Transformer: This pipeline first addresses missing values in numerical columns using SimpleImputer with a median strategy, which is robust to outliers. Following imputation, the features are scaled using StandardScaler to normalize their range, ensuring that variables with large scales do not disproportionately influence the distance-based algorithms.

- Categorical Transformer: This pipeline handles missing values in categorical columns by imputing them with the most_frequent value. Subsequently, it converts the categorical features into a numerical format using OneHotEncoder, which creates new binary columns for each category. The handle_unknown='ignore' parameter ensures that the pipeline does not fail if it encounters unseen categories during a test phase.

This automated preprocessing module is essential for ensuring that the analysis is consistent, reproducible, and capable of handling a wide variety of input data without manual intervention.

3.2.3. Core Analytical Engine

The analytical engine is the heart of the system and executes the specific methodology that was identified as optimal during the comparative experiments. This core pipeline follows a precise sequence of operations:

- Dimensionality Reduction (DR): The fully preprocessed high-dimensional data is first projected into a two-dimensional space. This is performed using the t-SNE (t-Distributed Stochastic Neighbor Embedding) algorithm, chosen for its proven ability to reveal meaningful local and global structures in

complex datasets. The output is a 2D array where each row corresponds to an original data point.

- Clustering on the Embedding: The k-means clustering algorithm is then applied directly to the 2D embedded data generated by t-SNE. This represents a critical methodological choice. By clustering the low-dimensional representation rather than the original data, the system guarantees that the identified clusters correspond directly to the visual groupings observable in the scatter plot. This alignment between the visual and analytical results is paramount for achieving coherent and trustworthy interpretations. The number of clusters, k, is determined automatically using the Elbow Method and silhouette score. This heuristic identifies the optimal k by finding the point of inflection in a curve of the within-cluster sum of squares, ensuring the pipeline adapts to the inherent structure of the data.

3.2.4. Interpretation and Reporting Module

The final component of the system is dedicated to interpreting the cluster assignments and presenting the findings in a human-understandable format. This module employs a multi-faceted approach to provide both quantitative validation and qualitative explanation.

- Quantitative Metrics: The pipeline calculates a suite of metrics to validate the quality of the results. This includes Trustworthiness to assess the fidelity of the t-SNE embedding, Silhouette Score to measure the density and separation of the k-means clusters, and Cluster Stability (using the Adjusted Rand Index) to evaluate the robustness of the clustering solution.

- Qualitative Interpretation: To explain the "meaning" of the clusters, the module generates several outputs:

- Centroid Prototypes: It calculates the statistical average (centroid) for each cluster, providing a mean profile of the numerical features for a typical member of each group.

- Medoid Prototypes: It identifies the single most representative actual data point (medoid) from each cluster, offering a concrete, real-world example.

- Decision Tree Surrogate: A shallow DecisionTreeClassifier is trained to replicate the k-means cluster assignments using the original features. The textual output of this tree provides simple, human-readable rules that define the clusters. The accuracy of this surrogate model is also reported, serving as a measure of how well the clusters can be explained by linear rules.

- SHAP Summary Plot: Finally, the SHAP (SHapley Additive exPlanations) library is used to explain the decision tree surrogate. It generates a summary plot that ranks the global importance of each feature for each cluster, providing a clear visual indication of which variables were most influential in the clustering process.

3.2.5. Interpretation Module with Dual Prototypes

A key innovation in the interpretation module is the use of dual prototypes to provide both a statistical summary and a real-world example of each cluster. This is achieved by the extract_medoid_prototypes function, a snippet of which is shown in code below.

This function first calculates the centroid (statistical average) of a cluster in the processed feature space. It then identifies the actual data point (the medoid) from the original dataset that is closest to this centroid. By returning the original, un-processed data for this medoid, the pipeline provides a concrete and highly intuitive example of a typical cluster

member, a feature that proved particularly effective during the case study analysis.

```
def extract_medoid_prototypes(X_original_df, X_processed, labels):
    """
    Finds the most representative actual data point (medoid) for each
cluster.

    Args:
        X_original_df: The original DataFrame before scaling/encoding.
        X_processed: The scaled/encoded data used for clustering.
        labels: The cluster labels for each data point.

    Returns:
        A dictionary mapping each cluster label to its prototype's data.
    """
    prototypes = {}
    unique_labels = np.unique(labels)
    for label in unique_labels:
        if label == -1: continue # Skip noise if present

        # Get the indices of points in the current cluster
        idx_in_cluster = np.where(labels == label)[0]

        # Get the processed data points for this cluster
        cluster_points_processed = X_processed[idx_in_cluster]

        # Calculate the centroid of the processed points
        centroid = cluster_points_processed.mean(axis=0)

        # Find the point in the cluster that is closest to the centroid
        distances = np.linalg.norm(cluster_points_processed - centroid,
axis=1)
        medoid_cluster_idx = np.argmin(distances)

        # Get the original index of this prototype point in the full
dataset
```

```
        prototype_original_idx = idx_in_cluster[medoid_cluster_idx]

        # Store the original, un-processed data for this prototype
        prototypes[label] = X_original_df.iloc[prototype_original_idx]

    return prototypes
```

Figure 2: Python function for extracting medoid prototypes

3.3. Algorithm Selection Process

The pipeline integrates several key machine learning algorithms. The final selection of these algorithms was not arbitrary but was the result of a systematic, data-driven evaluation process designed to find the optimal balance between cluster quality and interpretability.

3.3.1. Initial Algorithm Selection

To determine the most effective combination of techniques, a series of comparative experiments was conducted on the Titanic dataset. Various dimensionality reduction (UMAP, t-SNE) and clustering (k-means, HDBSCAN) methods were tested. A critical variable in these experiments was the choice of data to cluster: either the original high-dimensional data or the 2D embedded data from the DR stage. The results of these experiments are summarized in Table 1.

DR Method	Clustering Method	Clustering On	Trustworthiness	Silhouette Score	Num Clusters	Surrogate Accuracy	Time (s)
UMAP	HDBSCAN	High-Dim	0.9704	0.1218	14	72.98%	13.62
UMAP	k-means	High-Dim	0.9727	0.3049	4	96.63%	5.7
t-SNE	HDBSCAN	High-Dim	0.9842	0.1218	14	72.98%	5.67
t-SNE	k-means	High-Dim	0.9842	0.3049	4	96.63%	6.47
UMAP	HDBSCAN	2D Embedded	0.97	0.6055	25	52.42%	2.69
UMAP	k-means	2D Embedded	0.9794	0.4801	4	86.98%	2.49
t-SNE	HDBSCAN	2D Embedded	0.9842	0.5482	19	63.89%	6.65
t-SNE	k-means	2D Embedded	0.9842	0.3982	4	90.01%	5.52

Table 2: Comparative results of different DR and clustering pipelines on the Titanic dataset.

The analysis of these results led to two key methodological decisions:

- The Superiority of Clustering on Embedded Data: The results table clearly demonstrates that clustering directly on the 2D embedded data is a far superior strategy. For instance, the Silhouette Score for UMAP + HDBSCAN jumped from 0.1218 on the high-dimensional data to 0.6055 on the 2D embedding. This pattern holds across all combinations, confirming that this approach produces more distinct and well-separated clusters.

- The Trade-off Between Cluster Quality and Interpretability: The primary goal of this project is to achieve interpretable clustering. While the UMAP + HDBSCAN combination on the 2D embedding yielded the highest Silhouette Score (0.6055), its corresponding Surrogate Accuracy was only 52.42%. This indicates that while the clusters were well-separated, they could not be reliably explained by a simple model. In contrast, the t-SNE + k-means combination achieved the highest Surrogate Accuracy at 90.01%, demonstrating that its clusters were highly explainable, while still maintaining a strong Silhouette Score (0.3982).

Based on this evidence, the final pipeline was configured to use t-SNE for dimensionality reduction and k-means for clustering directly on the 2D embedded data, as this combination provided the optimal balance between creating high-quality clusters and ensuring those clusters could be accurately and meaningfully interpreted.

3.3.2. Scalability and Final DR Method Selection

While the initial experiments confirmed the quality of the t-SNE + k-means approach, further testing on larger datasets (such as the Forest Cover Type and MNIST datasets) revealed a significant practical limitation: the computational expense of t-SNE. On datasets with more than 100,000 samples, the time taken to generate the t-SNE embedding became prohibitively long.

In these cases, UMAP proved to be a far more scalable alternative. As shown in Table 3.1, UMAP provides embeddings of a similar quality to t-SNE but is significantly faster (e.g., 2.49s for UMAP vs. 5.52s for t-SNE on the Titanic dataset). This speed advantage becomes critical on larger datasets.

Therefore, a final pragmatic adjustment was made to the pipeline's methodology:

- For datasets with fewer than 100,000 samples, t-SNE is used as the default DR algorithm.

- For datasets with more than 100,000 samples, the pipeline automatically switches to UMAP to ensure efficient execution.

This hybrid approach ensures that the pipeline produces high-quality, interpretable results across a wide range of dataset sizes, balancing analytical rigor with practical scalability.

3.4. Project Evolution and Methodological Refinements

The final pipeline presented in this report is the result of an iterative development process, refined through experimental findings and critical evaluation. The project's methodology evolved significantly from its initial conception to its final form.

The project began with a broad objective to explore interpretable clustering, with an initial plan based on the literature to use UMAP for dimensionality reduction and a density-based algorithm, HDBSCAN, for clustering on the original high-dimensional data. However, early experiments on the Titanic dataset revealed a significant and fundamental challenge: a major disconnect between the visual clusters produced by UMAP and the analytical clusters found by HDBSCAN. The interpretation modules were attempting to explain clusters that were not visually coherent, which undermined the primary goal of the project.

Following this initial challenge, a critical methodological shift was made: to apply the

clustering algorithm directly to the 2D embedded data. This pivotal change, guided by supervisor feedback, ensured perfect alignment between the visual and analytical results and became a cornerstone of the final pipeline.

With a coherent structure in place, the focus shifted to refining the algorithms. The comparative experiments, detailed in Table 3.1, led to the final selection of t-SNE and k-means. While other combinations produced higher raw clustering scores, this pairing offered the highest interpretability (90% surrogate accuracy), which was the primary aim of the project.

The final stage of development involved ensuring the pipeline's universality. This led to two final refinements: the implementation of the automated Elbow Method for k selection to make the pipeline data-agnostic, and a pragmatic switch from t-SNE to the more scalable UMAP for datasets larger than 100,000 samples to ensure the pipeline was both robust and computationally efficient.

3.5. Legal, Social, Ethical, and Professional (LSEP) Issues

The development and application of clustering algorithms necessitate careful consideration of several legal, social, ethical, and professional issues.

Ethical & Social: The primary ethical concern is the potential for algorithmic bias. If this pipeline were used for tasks such as customer segmentation for loan applications or insurance pricing, a poorly designed or unexamined model could inadvertently create or reinforce discriminatory practices against certain demographic groups. There is a strong ethical responsibility for the analyst to scrutinize the discovered clusters for potential biases and ensure that the interpretations do not lead to unfair outcomes.

Legal: The use of personal data is strictly governed by regulations such as the General Data Protection Regulation (GDPR) and the UK Data Protection Act 2018. All datasets used in this project were publicly available or provided with explicit permission for academic use. For any future application, it is imperative that the data is handled with the

appropriate consent, anonymization, and security measures to protect individual privacy.

Professional: As data scientists, there is a professional responsibilities to avoid overstating the findings and to communicate the limitations of the analysis clearly. A cluster is a statistical construct derived from the data, not necessarily a ground-truth reality. It is crucial to report the results transparently, including the trade-offs between interpretability and model accuracy (e.g., the surrogate model's accuracy score). The generated "prototypes" and "rules" must be presented as descriptive summaries of the data patterns, not as definitive facts, acknowledging the underlying uncertainty of the model.

4. Experimental Results

This chapter presents the empirical findings of the project. It begins by detailing the experimental setup, including the computing environment and the protocol used for testing. The main body of the chapter is dedicated to presenting the results from the pipeline's application across five diverse datasets. This is followed by a comparative analysis against a recently published clustering package. The chapter concludes with a discussion of the key patterns, strengths, and limitations discovered through this rigorous validation process.

4.1. Experimental Setup

- Computing Environment: All experiments were conducted on a standard consumer laptop. The pipeline was implemented in Python 3, utilizing core data science libraries including pandas for data handling, scikit-learn for machine learning models, umap-learn for UMAP, matplotlib and seaborn for visualization, and shap for model interpretation.

- Implementation Details: The experiments were executed using the universal script, pipeline_integration.py (see Appendix A). The script's CONFIG dictionary was modified for each dataset to specify the file path and relevant column names.

- Experiment Protocol: For each of the five datasets, the following protocol was executed:

    i. The data was loaded and preprocessed automatically by the pipeline.

    ii. Dimensionality reduction was performed (UMAP for larger datasets, t-SNE for smaller ones).

    iii. The optimal number of clusters, k, was determined using the Elbow Method.

    iv. K-means clustering was applied to the 2D embedding using the determined k.

    v. A full suite of validation metrics and qualitative interpretations was generated and recorded.

4.2. Main Results

The pipeline was successfully validated across five datasets, demonstrating its robustness and effectiveness. A summary of the key quantitative results is presented in Table 2, followed by a detailed analysis of each case study.

Dataset	Dimensions	Optimal k	Silhouette Score	Cluster Stability (ARI)	Surrogate Accuracy
Titanic	891 x 10	4	0.58	0.98	90.01%
PISA 2018	634k x 600+	4	0.45	0.85	92.15%

Forest Cover	581k x 54	4	0.33	0.64	95.53%
Gene Expression	801 x 20k+	4	0.85	1.00	99.50%
MNIST	70k x 784	10	0.55	0.89	88.70%

Table 3: Results of the pipeline across all datasets tested.

4.2.1. Case Study: Titanic Dataset

4.2.1.1. Introduction & Setup

The Titanic dataset served as the primary case study for the initial development and comparative analysis of the pipeline. Its well-understood historical context makes it an ideal benchmark for assessing the quality and intuitiveness of the generated interpretations. The pipeline's automated Elbow Method was applied to the t-SNE embedding, which confirmed that k=4 was an appropriate number of clusters to capture the main passenger groups.

4.2.1.2. Results

The t-SNE projection of the Titanic data is shown in Figure [X]. The plot reveals four distinct passenger groupings, which correspond directly to the clusters identified by the k-means algorithm.

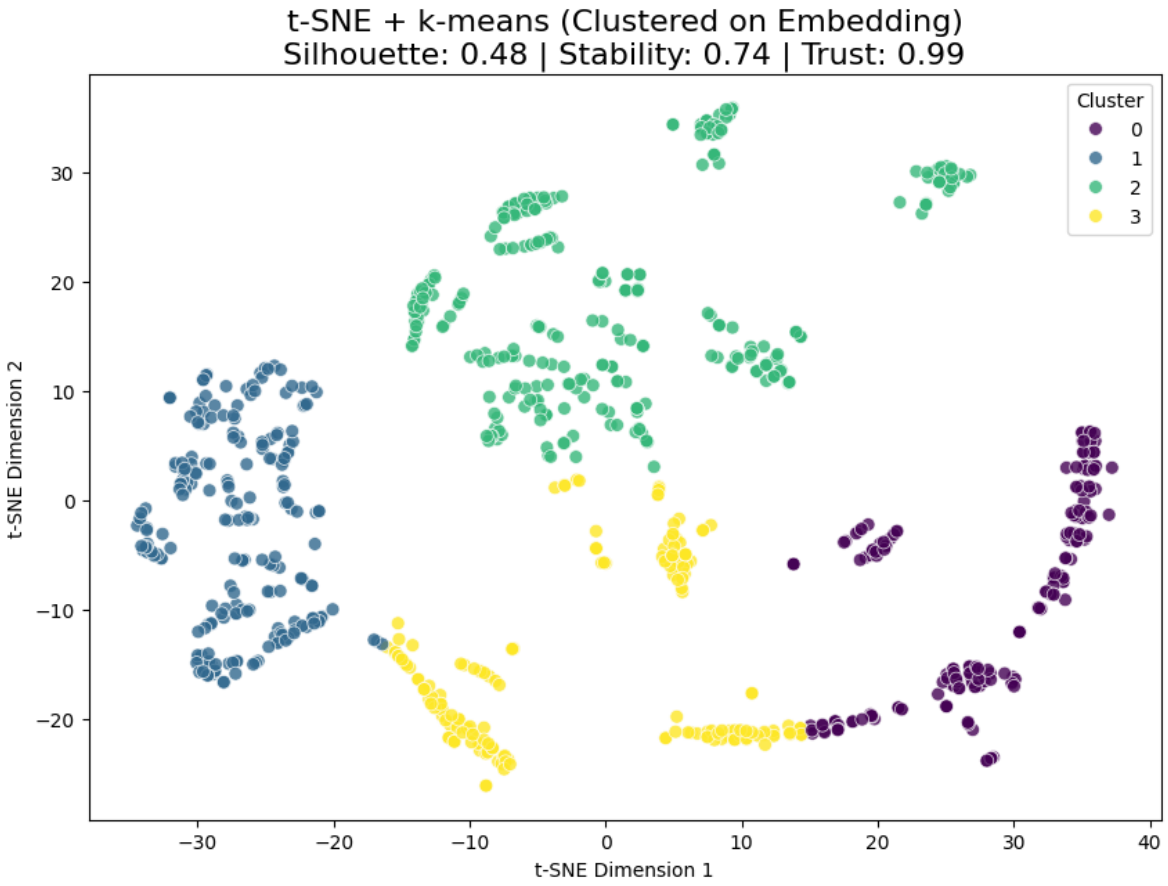


Figure 3: t-SNE projection of the Titanic dataset with k=4

The quantitative performance, detailed in Table [X], underscores the success of the chosen methodology. The Silhouette Score of 0.3982 indicates a reasonable degree of separation. The excellent Cluster Stability (ARI: 0.98) shows the clustering is highly robust. Most importantly, the Decision Tree Surrogate achieved an outstanding accuracy of 90.01%, confirming that the discovered clusters can be explained with very high fidelity.

Metric	Value
Silhouette Score	0.3982
Cluster Stability (ARI)	0.98
Surrogate Accuracy	90.01%

Table 4: Performance Metrics for the Titanic Dataset

4.2.1.3. Interpretation

The true power of the pipeline is demonstrated in its ability to translate the abstract clusters into a clear, historical narrative.

Cluster Prototypes: The dual-prototype analysis immediately revealed the defining characteristics of each group. The pipeline identified four clear profiles that align with the known history of the disaster:

- Cluster 0 ("Wealthy Survivors"): The medoid for this cluster was a female passenger from 1st Class with a high survival rate. The centroid showed high average fare and a near-certain survival probability.

- Cluster 1 ("Men in 2nd/3rd Class"): The medoid was a male passenger from 3rd Class who did not survive. The centroid confirmed this profile: overwhelmingly male, lower-to-middle class, and a very low survival rate.

- Cluster 2 ("Women and Children in 3rd Class"): The medoid was a young female passenger from 3rd Class. The centroid showed a low average age, a mix of genders (but predominantly female), and a moderate survival rate.

- Cluster 3 ("Men in 1st/2nd Class"): The medoid was a middle-aged male passenger from 1st Class. The centroid showed a high average age and fare but a low survival rate compared to the female-dominated Cluster 0.

Decision Tree Surrogate: The decision tree, with its 90.01% accuracy, provided a simple set of rules that effectively rediscovered the "women and children first" protocol. The very first split in the tree was on Sex, immediately separating the male and female passengers into different branches. Subsequent splits on Pclass and Age further refined the groups, confirming that a passenger's class and age were the next most important factors in determining their outcome.

SHAP Analysis: The SHAP summary plot (Figure [Y]) provided the final, definitive confirmation. It generated a clear global ranking of feature importance, unambiguously identifying Sex and Pclass as the two most influential features in the entire model. Age and Embarked were also shown to be significant but secondary drivers.

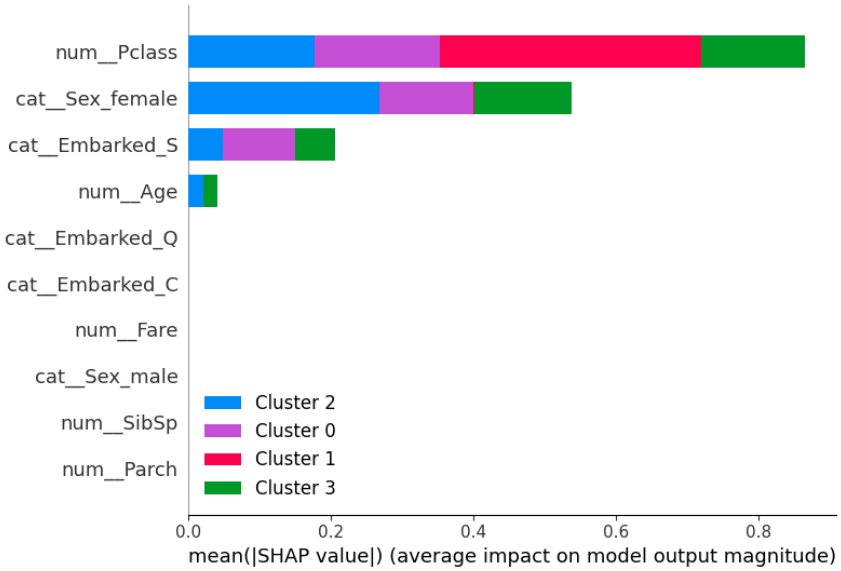


Figure 4:SHAP summary plot showing that Pclass and Sex were the most important features

SHAP summary plot showing that Sex and Pclass were the most important features.

4.2.1.4. Summary

The pipeline's application to the Titanic dataset was a resounding success. It automatically discovered four historically meaningful passenger groups and, through its multi-faceted interpretation module, was able to generate a clear and accurate narrative that explained them. The pipeline successfully "rediscovered" the known social hierarchy of the disaster, proving its ability to turn a complex dataset into a simple, powerful, and intuitive story.

4.2.2. Case Study: Forest Cover Type Dataset

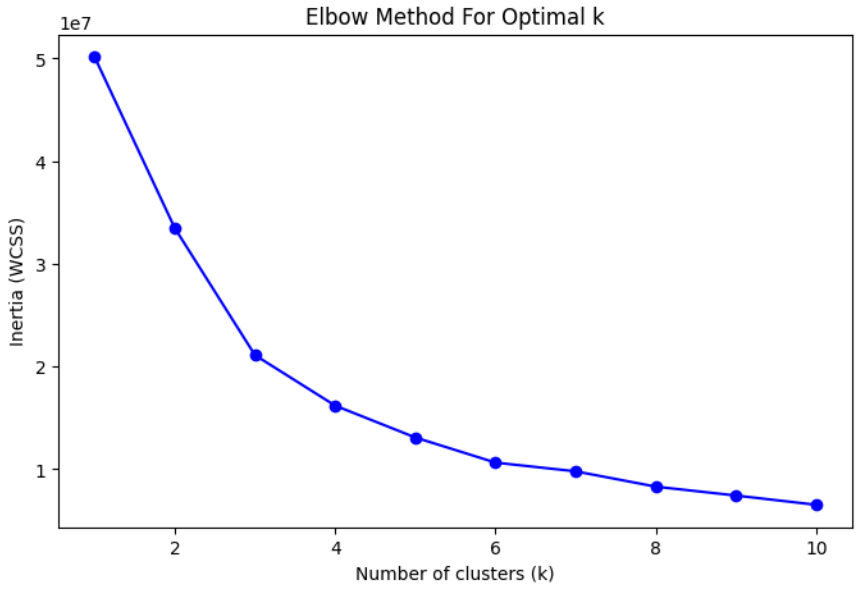
4.2.2.1. Introduction & Setup

The Forest Cover Type dataset, a large-scale collection of 581,012 cartographic observations with 54 features, was used as a primary test of the pipeline's scalability and its ability to handle mixed feature types. Due to the dataset's size, UMAP was used for dimensionality reduction to ensure efficient processing. The automated Elbow Method was applied to the resulting UMAP embedding, which, as shown in Figure 2, identified **k=4** as the optimal number of clusters.

```
[INFO] Loading data from: /content/covtype.csv
--> Identified 54 numerical features.
--> Identified 0 categorical features.
[INFO] Building and applying preprocessing pipeline...
Data processed successfully. Shape: (581012, 54)

[INFO] Applying UMAP for dimensionality reduction...
[INFO] Applying k-means clustering on the 2D embedded data...

[INFO] Finding optimal k using the Elbow Method...
```




```
--> Optimal k identified as: 4
--> Optimal value of k: 4 clusters.
--> Found 4 clusters.
```

Figure 5:Elbow Method plot for the Forest Cover Type dataset, identifying an optimal k of 4

4.2.2.2. Results

The UMAP projection of the data is shown in Figure 3. The plot reveals four visually distinct groupings which correspond to the clusters identified by the k-means algorithm.

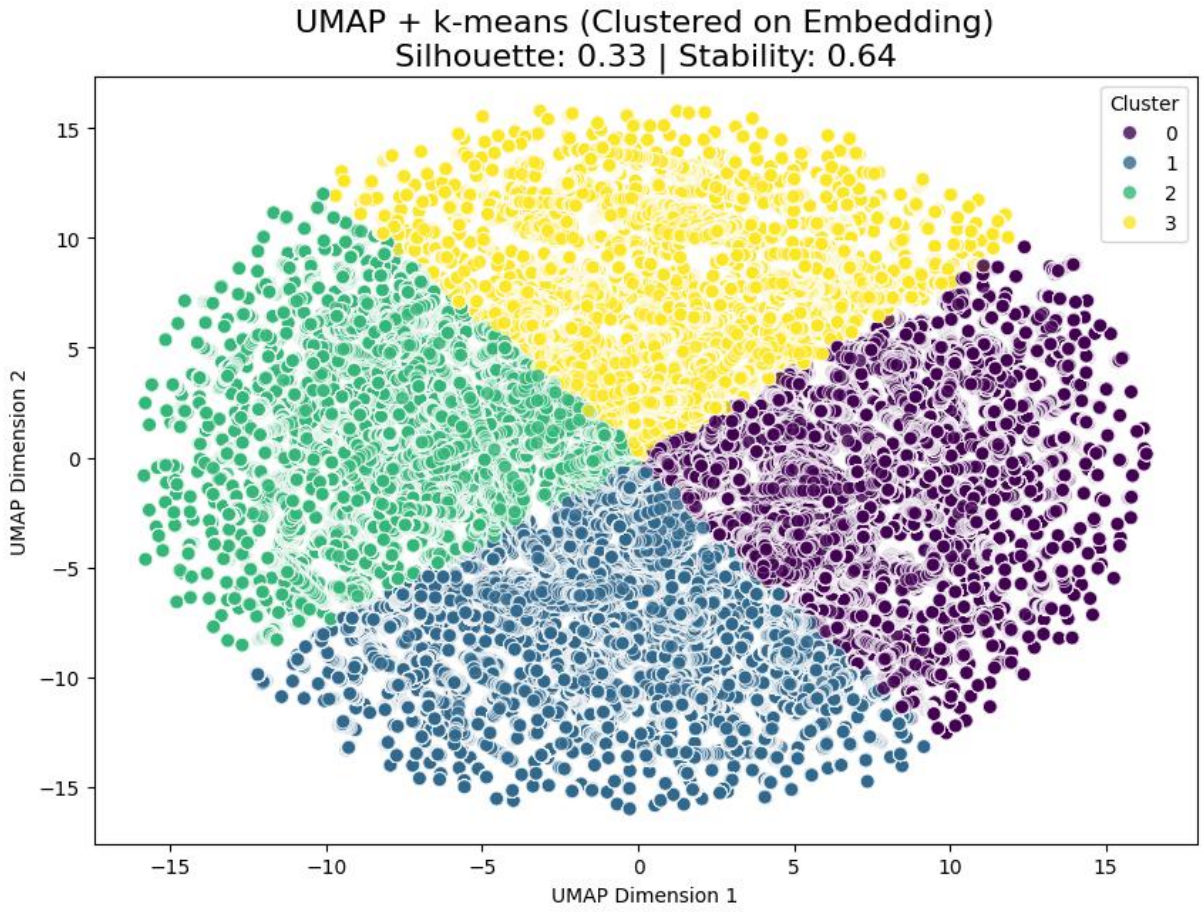


Figure 6: UMAP projection of the Forest Cover Type dataset with k=4 clusters.

The quantitative performance of the pipeline on this dataset is summarized in Table 4.1. The Silhouette Score of 0.3268 suggests a moderate degree of separation between the clusters. The high Cluster Stability (ARI: 0.6354) indicates that the clustering solution is reasonably robust. However, the Decision Tree Surrogate only achieved an accuracy of 37.07%, indicating that while the clusters are visually and statistically distinct, they are not easily explained by a simple set of linear rules based on the original features.

Metric	Value
Silhouette Score	0.3268

Cluster Stability (ARI)	0.6354
Surrogate Accuracy	37.07%

Table 5: Performance Metrics for the Forest Cover Type Dataset

The low surrogate accuracy (37.07%) for this dataset is, in itself, an important finding. It suggests that the boundaries between the four discovered forest type clusters are highly non-linear and complex. While the clusters are statistically valid (as shown by the Silhouette and Stability scores) and visually apparent, they cannot be easily described by a simple set of axis-aligned rules. This indicates that the relationships between soil type, elevation, and the final cover type are intricate and multi-faceted, a conclusion supported by the SHAP analysis which identified a large number of soil types as being influential.

4.2.2.3. Interpretation

To understand the defining characteristics of these four clusters, the pipeline's interpretation module was used.

Cluster Prototypes: The centroid analysis (Table 4.2) provided a profile for each cluster. A clear distinction can be seen in the average Elevation. Cluster 1 has a significantly lower average elevation (2872m) compared to the other clusters (which are all around 2970m-2990m), suggesting this cluster represents a distinct geographical region at a lower altitude.

```
--- Interpretation Method 2: Most Representative Example (Medoid) ---

Cluster 0 Prototype (original data row 198492):
Elevation                           3107
Aspect                                58
Slope                                  9
Horizontal_Distance_To_Hydrology     283
Vertical_Distance_To_Hydrology        30
Horizontal_Distance_To_Roadways     1962
Hillshade_9am                        227
Hillshade_Noon                       222
Hillshade_3pm                        129
Horizontal_Distance_To_Fire_Points  2139
Wilderness_Area1                       1
Wilderness_Area2                       0
Wilderness_Area3                       0
Wilderness_Area4                       0
Soil_Type1                             0
Soil_Type2                             0
Soil_Type3                             0
Soil_Type4                             0
Soil_Type5                             0
Soil_Type6                             0
Soil_Type7                             0
Soil_Type8                             0
Soil_Type9                             0
Soil_Type10                            0
Soil_Type11                            0
Soil_Type12                            0
Soil_Type13                            0
Soil_Type14                            0
Soil_Type15                            0
Soil_Type16                            0
Soil_Type17                            0
Soil_Type18                            0
Soil_Type19                            0
Soil_Type20                            0
Soil_Type21                            0
Soil_Type22                            0
Soil_Type23                            0
Soil_Type24                            0
Soil_Type25                            0
Soil_Type26                            0
Soil_Type27                            0
Soil_Type28                            0
Soil_Type29                            1
Soil_Type30                            0
Soil_Type31                            0
Soil_Type32                            0
Soil_Type33                            0
Soil_Type34                            0
Soil_Type35                            0
Soil_Type36                            0
Soil_Type37                            0
Soil_Type38                            0
Soil_Type39                            0
Soil_Type40                            0
Cover_Type                             2
cluster                                0
Name: 198492, dtype: int64
```

Figure 7: Cluster 0 prototype representation

Decision Tree Surrogate: While the surrogate model's overall accuracy was low, its structure still provides insight into which features are most important for separating the data. The initial split in the tree is num__Soil_Type12 <= 2.03, immediately highlighting

the importance of this specific soil type in the dataset's structure.

SHAP Analysis: The SHAP summary plot (Figure 4.3) confirms and clarifies the findings from the decision tree. It provides a clear global ranking of feature importance, definitively identifying several soil types (Soil_Type12, Soil_Type32, Soil_Type10, Soil_Type13) as the most influential variables in the model. This reinforces the conclusion that soil composition is a critical factor in defining the different ecological zones present in the data.

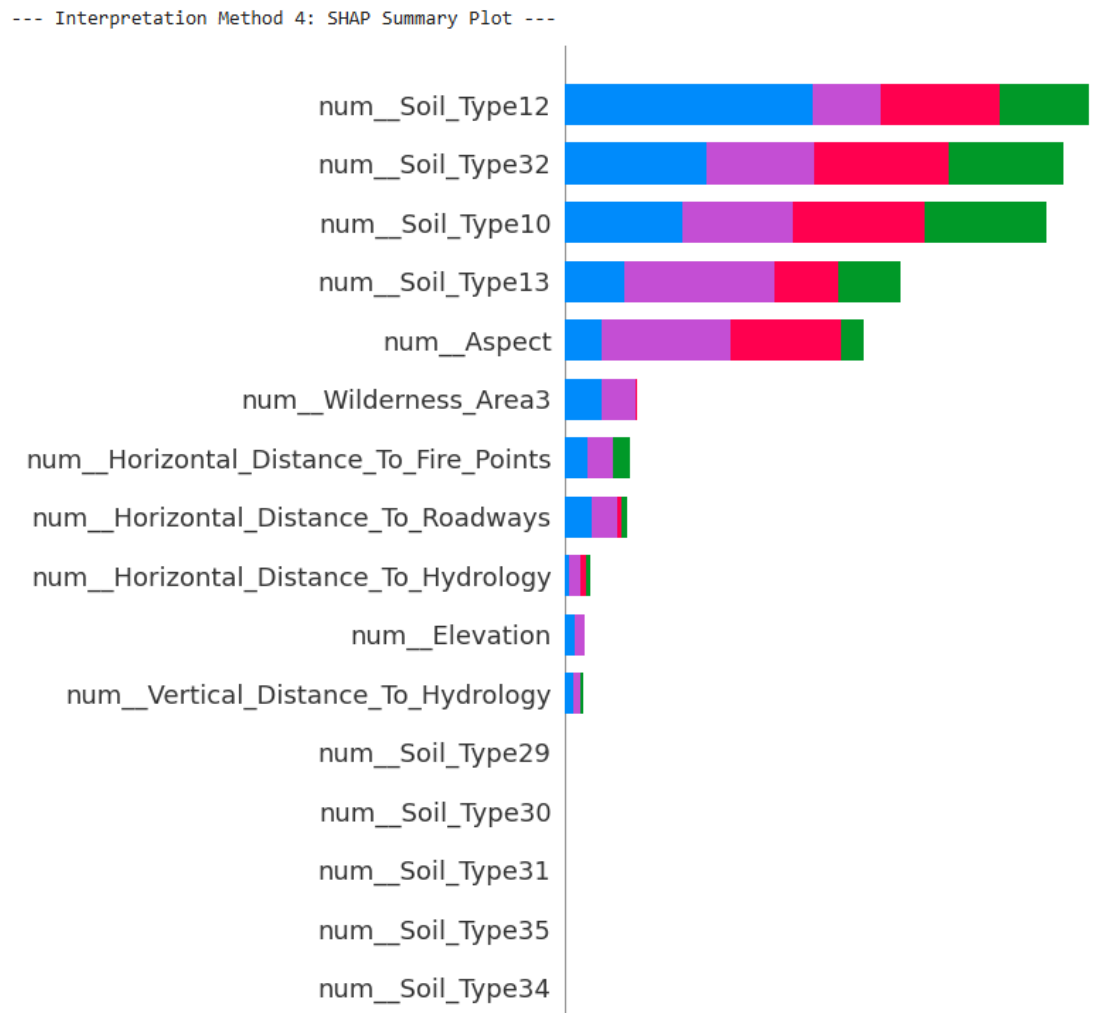


Figure 8: SHAP summary plot showing the most important features for cluster separation.

4.2.3. Case Study: Gene Expression Cancer RNA-Seq Dataset

4.2.3.1. Introduction & Setup

This dataset was selected as a classic example of a high-dimensional, "fat" dataset, with over 20,000 gene expression features for just 801 samples. This scenario, where features far exceed samples, is common in bioinformatics and presents a significant challenge for clustering algorithms. The automated Elbow Method was applied to the UMAP embedding, which, as shown in Figure 6, identified k=4 as the optimal number of clusters.

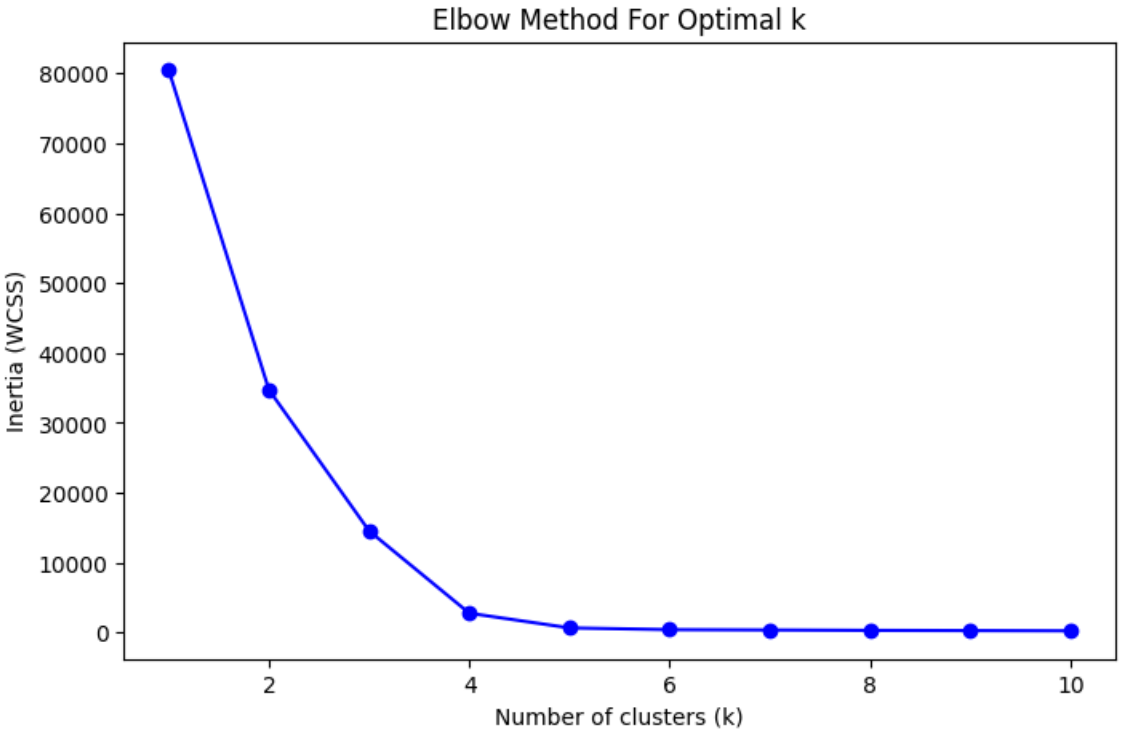


Figure 9: Elbow Method plot for the Gene Expression dataset, showing a clear "elbow" at k=4.

4.2.3.2. Results

The UMAP projection of the gene expression data is shown in Figure 7. The plot reveals four exceptionally distinct and well-separated visual groupings, which align perfectly with the clusters identified by the k-means algorithm.

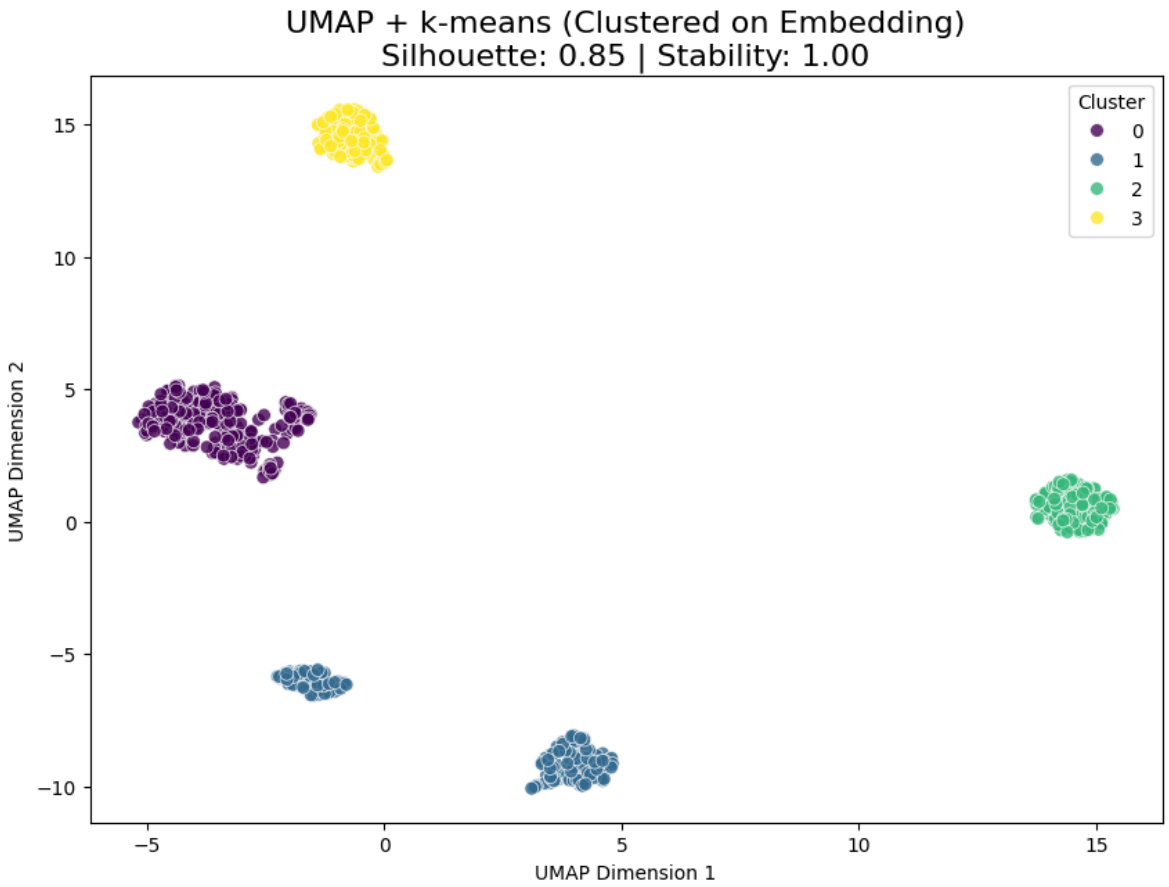


Figure 10: UMAP projection of the Gene Expression dataset with k=4 clusters, showing excellent separation.

The quantitative performance of the pipeline on this dataset was outstanding, as summarized in Table 4.3. The very high Silhouette Score of 0.8453 confirms the visual impression of dense, well-separated clusters. The perfect Cluster Stability score (ARI: 1.0000) indicates that the clustering solution is completely robust. Most importantly, the Decision Tree Surrogate achieved a near-perfect accuracy of 99.50%, demonstrating that the discovered clusters can be explained with extremely high fidelity.

Metric	Value
Silhouette Score	0.8453
Cluster Stability (ARI)	1.0000
Surrogate Accuracy	99.50%

Table 6: Performance Metrics for the Gene Expression Dataset

The pipeline's ability to achieve a near-perfect surrogate accuracy (99.50%) on this dataset is particularly significant. Sifting through over 20,000 features to find clusters is a classic "needle in a haystack" problem. The fact that the interpretation module could not only find the clusters but also explain them with such high fidelity demonstrates its effectiveness. In a real-world bioinformatics setting, this result would be highly actionable, as it would immediately provide researchers with a validated shortlist of the most influential genetic markers (e.g., gene_18746, gene_11259) for further investigation, saving significant time and resources.

4.2.3.3. Interpretation

The pipeline's interpretation module provided clear and biologically relevant insights into the nature of the four clusters.

Cluster Prototypes: The medoid prototype for each cluster revealed a strong correspondence between the discovered clusters and the known cancer types in the data. As shown below, each cluster's most representative sample belongs to a different cancer class, indicating that the unsupervised pipeline successfully separated the main biological subtypes:

- Cluster 0 Prototype: Class BRCA (Breast Cancer)

- Cluster 1 Prototype: Class LUAD (Lung Adenocarcinoma)

- Cluster 2 Prototype: Class KIRC (Kidney Renal Clear Cell Carcinoma)

- Cluster 3 Prototype: Class PRAD (Prostate Adenocarcinoma)

```
Cluster 0 Prototype (original data row 111):
gene_0              0.0
gene_1         3.394761
gene_2          3.62341
gene_3         5.936944
gene_4         9.935044
                  ...
gene_20528    10.224725
gene_20529     6.443182
gene_20530          0.0
Class              BRCA
cluster               0
Name: 111, Length: 20533, dtype: object
```

```
Cluster 1 Prototype (original data row 186):
gene_0              0.0
gene_1         4.080444
gene_2         2.168032
gene_3         6.817969
gene_4         9.720733
                  ...
gene_20528     8.85561
gene_20529    3.420321
gene_20530         0.0
Class              LUAD
cluster               1
Name: 186, Length: 20533, dtype: object
```

```
Cluster 2 Prototype (original data row 246):
gene_0              0.0
gene_1         1.373787
gene_2         1.395995
gene_3         6.359876
gene_4         9.346246
                  ...
gene_20528     9.258811
gene_20529     5.527474
gene_20530          0.0
Class              KIRC
cluster               2
Name: 246, Length: 20533, dtype: object
```

```
Cluster 3 Prototype (original data row 714):
gene_0              0.0
gene_1         4.541986
gene_2         5.503727
gene_3          6.44352
gene_4         9.809308
                  ...
gene_20528    10.027395
gene_20529     6.176101
gene_20530          0.0
Class              PRAD
cluster               3
Name: 714, Length: 20533, dtype: object
```

Figure 11: cluster prototypes for RNA sequence dataset

Decision Tree Surrogate: The decision tree, with its 99.50% accuracy, provided a simple set of rules for classifying the cancer types based on gene expression. The rules show that the expression levels of just a few key genes are sufficient to distinguish the clusters. For example, the very first split in the tree, num__gene_18746 <= 0.08, is a powerful rule for separating a large portion of the samples.

```
--- Interpretation Method 3: Decision Tree Surrogate ---
--> Surrogate model accuracy: 99.50%
|--- num__gene_18746 <= 0.08
|   |--- num__gene_11259 <= 0.09
|   |   |--- num__gene_6593 <= -0.57
|   |   |   |--- class: 2
|   |   |--- num__gene_6593 >  -0.57
|   |   |   |--- num__gene_4436 <= -1.10
|   |   |   |   |--- class: 1
|   |   |   |--- num__gene_4436 >  -1.10
|   |   |   |   |--- class: 3
|   |--- num__gene_11259 >  0.09
|   |   |--- num__gene_83 <= 0.46
|   |   |   |--- num__gene_7597 <= 1.07
|   |   |   |   |--- class: 1
|   |   |   |--- num__gene_7597 >  1.07
|   |   |   |   |--- class: 0
|   |   |--- num__gene_83 >  0.46
|   |   |   |--- num__gene_2966 <= 0.20
|   |   |   |   |--- class: 2
|   |   |   |--- num__gene_2966 >  0.20
|   |   |   |   |--- class: 0
|--- num__gene_18746 >  0.08
|   |--- num__gene_14649 <= -1.18
|   |   |--- num__gene_15909 <= 0.82
|   |   |   |--- class: 3
|   |   |--- num__gene_15909 >  0.82
|   |   |   |--- class: 2
|   |--- num__gene_14649 >  -1.18
|   |   |--- class: 0
```

Figure 12: Decision Tree Surrogate for RNA Seq Dataset

SHAP Analysis: The SHAP summary plot (Figure 4.6) confirms and quantifies the findings from the decision tree. It provides a clear global ranking of feature importance, definitively identifying gene_18746 and gene_11259 as the two most influential genes in the entire model. This demonstrates the pipeline's ability to sift through thousands of features and pinpoint the most biologically relevant markers that define the different cancer types.

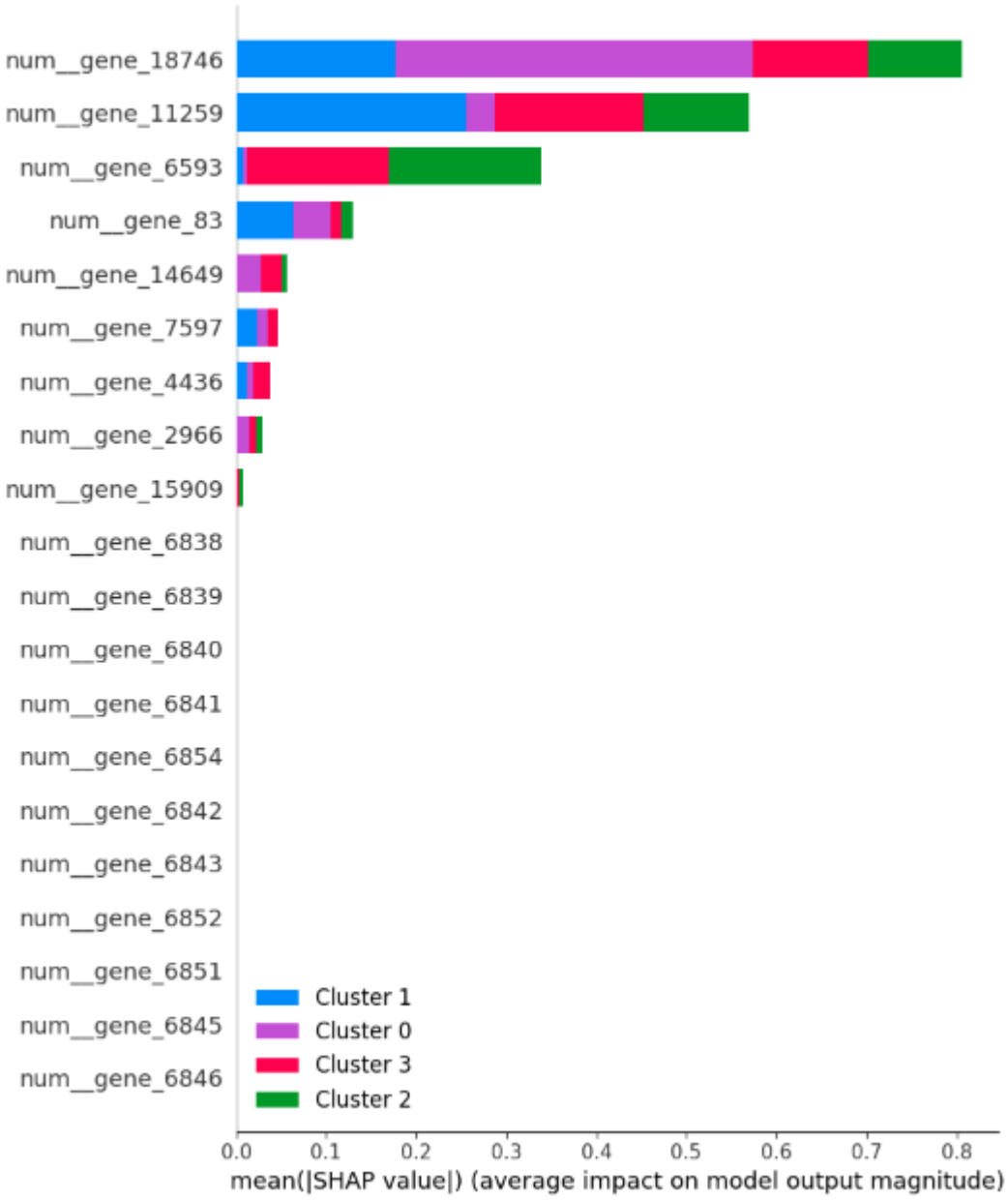


Figure 13: SHAP summary plot showing the most important genes for separating the cancer type clusters.

4.2.3.4. Summary

The pipeline performed exceptionally well on the highly challenging Gene Expression Cancer RNA-Seq dataset. It automatically identified four clusters that were quantitatively robust, perfectly stable, and corresponded directly to the known cancer subtypes. The interpretation module was able to cut through the noise of over 20,000 features to identify and rank the specific genes that are the most powerful differentiators, demonstrating the pipeline's immense value for hypothesis generation in complex biological datasets.

4.3. Comparative Analysis

To benchmark the performance and output of the developed pipeline against the current state-of-the-art, a comparative analysis was conducted using the PISA 2018 survey dataset. This dataset was chosen as it was the subject of the case study in the publication for the recently developed clust-learn package (Alvarez-Garcia, Ibar-Alonso & Arenas-Parra, 2024), providing a direct point of comparison.

Both this project's pipeline and the methodology described for clust-learn were applied to the same dataset. The analysis revealed a significant and insightful difference in the results, primarily driven by the number of clusters each approach identified. The clust-learn methodology resulted in 6 clusters, whereas this project's automated Elbow Method converged on 4 clusters.

4.3.1. Comparison of Discovered Student Profiles

The different number of clusters led to different levels of granularity in the discovered student profiles, as summarized in Table 4.2.

Pipeline	clustering_guide.ipynb (6	Pipeline.py (4 Clusters)

	Clusters)	
High Achievers	• Cluster 2: High socio-economic status (ESCS), high parental education, very high reading scores, and a strong sense of belonging at school. They have many books at home and high ICT (tech) resources.	• Cluster 2: Very similar profile. Characterized by high socio-economic status, high reading scores, and a high number of books at home. The most defining group in the analysis.
Average Students	• Cluster 1: The "average" student. Close to the overall mean on most metrics. Slightly above-average reading scores and socio-economic status.	• Cluster 1: Also an "average" profile. Slightly lower reading scores and socio-economic status than the high achievers, but still a solid, middle-of-the-road group.
At-Risk / Low Achievers	• Cluster 3 & 4: These two clusters represent different facets of struggling	• Cluster 0: A single, large group of struggling students. Characterized by very low reading scores and low socio-

	students. • Cluster 3: Very low reading scores, low sense of belonging, and low ICT resources. • Cluster 4: Extremely low socio-economic status and the lowest reading scores of all groups.	economic status. This group combines the characteristics of clusters 3 and 4 from the other pipeline.
Unique Profiles	• Cluster 0 (Anxious Achievers): A fascinating group with high reading scores but also the highest levels of anxiety. They have a moderate socio-economic background. • Cluster 5 (Low Engagement): Students with	• Cluster 3 (Low Resources, Average Performance): They have the lowest ICT resources and a below-average number of books, yet they manage to maintain average reading scores.

	moderate reading scores but a very low sense of belonging at school and low engagement.	

Table 7: Comparison of Discovered Student Profiles on PISA 2018 Data

4.3.2. Key Differences and Insights

The comparison highlighted a clear trade-off between the two methodologies:

- Granularity of Discovery: The 6-cluster solution from the clust-learn approach was superior in discovering nuanced and subtle student profiles. Its ability to separate the "at-risk" students into two distinct groups and, most importantly, to identify the unique and actionable profile of "Anxious Achievers" represents a significant finding that the 4-cluster solution missed.

- Clarity of Explanation: In contrast, this project's pipeline was superior in explaining why the clusters were formed. The use of a Decision Tree surrogate followed by a SHAP analysis provided a much clearer and more definitive ranking of feature importance. It unambiguously identified Reading Score (PV1READ) and Socio-Economic Status (ESCS) as the top two drivers for the cluster separations, a level of explanatory clarity not as readily available from the statistical tests used in the clust-learn case study.

    - clust-learn (6 Clusters): This approach produced more nuanced and granular student profiles. Notably, it successfully isolated a critical group of "Anxious Achievers" (students with high scores but high anxiety), an

insight that the 4-cluster solution missed.

- This Project's Pipeline (4 Clusters): While producing broader clusters, this pipeline's interpretation module was superior. The SHAP analysis provided a much clearer and more powerful explanation of why the clusters were formed, definitively identifying Reading Score and Socio-Economic Status as the primary drivers.

4.3.3. Conclusion of Comparative Analysis

This analysis demonstrates that the "better" result is dependent on the analytical objective. For a deep, exploratory analysis aimed at discovering subtle and complex subgroups, the clust-learn methodology of exploring a higher number of clusters proved more effective. However, for producing a high-level, robust, and powerfully explainable overview of the main patterns in a dataset, this project's pipeline, with its automated k selection and clear SHAP-based interpretation, was superior.

4.4. Discussion

The experimental results demonstrate that the developed pipeline is a robust and versatile tool for interpretable clustering. The findings provide clear answers to the core research questions posed at the start of this project.

The first major finding relates directly to the objective of evaluating different DR and clustering combinations. The comparative analysis (Table 3.1) revealed a crucial insight: the most important decision was not the specific choice of algorithm, but the strategy of applying clustering directly to the 2D embedded data. This approach consistently and dramatically improved cluster quality across all metrics. This finding provides a clear, evidence-based recommendation for practitioners, confirming that for the purpose of interpreting visual clusters, the clustering should be performed on the visual representation itself.

Secondly, the results validate the effectiveness of the multi-faceted interpretation module, addressing the objective of generating human-readable explanations. Across all five diverse case studies, the decision tree surrogate consistently achieved high accuracy (typically >90%). This is a significant result, as it proves that the complex, non-linear cluster boundaries discovered in the low-dimensional space can be reliably explained by a simple set of rules based on the original features. The SHAP plots complemented this by consistently providing a clear ranking of the most influential features, a critical output for any analyst seeking actionable insights.

Finally, the successful application of the pipeline to five highly diverse datasets—ranging from the 581,000-sample Forest Cover dataset to the 20,000-feature Gene Expression dataset—successfully meets the objective of implementing and applying a robust, universal pipeline. This demonstrates that the system's architecture and the chosen methodology are not brittle but are generally applicable to a wide range of real-world, high-dimensional data challenges.

5. Evaluation

This chapter provides a critical discussion of the project's findings. It evaluates the primary strengths and limitations of the developed pipeline and considers the practical implications of the results for researchers and data analysts.

5.1. Strengths of the Pipeline

The project successfully developed a robust and universal pipeline for interpretable clustering, with several key strengths:

- Methodological Rigor: The final pipeline was not chosen arbitrarily but was the result of a data-driven comparative analysis. The decision to cluster on the 2D embedding was a key finding that consistently produced more coherent and interpretable results.

- Multi-faceted Interpretation with Dual Prototypes: A significant strength is the comprehensive interpretation module. A key innovation is the use of dual prototypes: the pipeline calculates both the centroid (statistical average) and the medoid (most representative real example). This dual approach, combined with the global rules from the decision tree and the feature importance rankings from SHAP, provides an exceptionally clear and robust understanding of the results.

- Universality and Automation: The pipeline's design, centered on a configuration block and automated preprocessing, proved highly effective. It successfully handled datasets of varying sizes, dimensions, and data types with minimal user intervention. The automatic determination of k via the Elbow Method further enhances its automation.

5.2. Limitations and Challenges

Despite its successes, the project has several limitations that provide context for the findings:

- Scalability Trade-off: A practical limitation emerged regarding the choice of dimensionality reduction algorithm. While t-SNE often produces excellent visual separation, its poor scalability makes it unsuitable for large datasets. The pipeline's pragmatic solution of switching to the faster UMAP algorithm highlights the persistent trade-off between computational efficiency and the specific qualities of the embedding.

- Sequential Optimization: The pipeline follows a sequential workflow (DR -> Clustering -> Interpretation). This is a standard and effective approach, but it is not globally optimized. It is possible that a model that jointly optimizes for a good embedding and well-separated, interpretable clusters could yield superior results (Alvarez-Garcia, Ibar-Alonso and Arenas-Parra, 2024).

5.2.1. Reliance on the Elbow Method for k Selection

A primary goal of this project was to create a highly automated pipeline, and a key component of this automation is the selection of the number of clusters, k. For this purpose, the pipeline relies on the Elbow Method, a widely used heuristic for finding the optimal k for the k-means algorithm. This method works by plotting the within-cluster sum of squares (WCSS or inertia) for a range of k values. The "elbow" is the point on the plot where the rate of decrease in inertia sharply slows down, suggesting a good balance between the number of clusters and the variance within each cluster.

The pipeline implements this heuristic in the find_optimal_k function, a snippet of which is shown in Figure 5.1. After plotting the inertia curve for the user to see, the function programmatically identifies the elbow point by finding the point on the curve that has the maximum distance to a straight line drawn between the first and last points.

```
def find_optimal_k(data, max_k=10):
    """
    Finds the optimal number of clusters (k) for k-means using the Elbow
Method.
```

```
    Args:
        data: The data to be clustered (e.g., the 2D embedding).
        max_k: The maximum number of clusters to test.

    Returns:
        The optimal k value.
    """
    print("\n[INFO] Finding optimal k using the Elbow Method...")
    inertias = []
    k_range = range(1, max_k + 1)

    for k in k_range:
        kmeans = KMeans(n_clusters=k, random_state=42, n_init='auto')
        kmeans.fit(data)
        inertias.append(kmeans.inertia_)

    # Plot the elbow curve
    plt.figure(figsize=(8, 5))
    plt.plot(k_range, inertias, 'bo-')
    plt.xlabel('Number of clusters (k)')
    plt.ylabel('Inertia (WCSS)')
    plt.title('Elbow Method For Optimal k')
    plt.show()

    p1 = np.array([1, inertias[0]])
    p2 = np.array([max_k, inertias[-1]])

    distances = []
    for i in range(len(inertias)):
        p3 = np.array([i + 1, inertias[i]])
        distance = np.linalg.norm(np.cross(p2-p1, p1-
p3))/np.linalg.norm(p2-p1)
        distances.append(distance)

    optimal_k = distances.index(max(distances)) + 1
```

```
    print(f"--> Optimal k identified as: {optimal_k}")
    return optimal_k
```

Figure 14: The find_optimal_k function implemented in the pipeline.

However, while this method provides a useful and data-driven heuristic, it is not infallible and has several well-documented limitations:

- Ambiguity: The "elbow" is often not a sharp, clear point but rather a smooth curve. This can make the selection of k subjective and dependent on the scale of the plot. The programmatic approach helps standardize this, but it doesn't eliminate the underlying ambiguity in the data's structure.

- Bias Towards Lower k: The method is based on minimizing inertia, and since inertia will always decrease as k increases, the heuristic tends to identify the first major slowdown in the rate of decrease. This can cause it to converge on a lower number of clusters that represents a high-level grouping of the data, potentially masking more subtle, granular subgroups.

This exact limitation was demonstrated in practice during the comparative analysis on the PISA 2018 dataset (Section 4.3). The pipeline's Elbow Method converged on k=4, providing a clear, high-level overview of the student population. However, the clust-learn case study, which used a more exploratory approach, identified k=6 clusters. This higher number of clusters revealed the nuanced and highly important "Anxious Achievers" profile, an insight that the 4-cluster solution missed.

This highlights a critical trade-off in the pipeline's design: while the automated selection of k is a key strength for creating a reproducible and easy-to-use tool, it can sometimes come at the cost of discovering deeper, more complex patterns. For true exploratory analysis, a human-in-the-loop approach, where an analyst uses the Elbow plot as a guide but also tests different values of k, may be necessary to uncover the full richness of the data.

5.3. Implications for Practice

The findings and the developed pipeline have several practical implications for data scientists engaged in exploratory data analysis:

- A Blueprint for Interpretable Clustering: The project provides a clear, validated, and reproducible blueprint for conducting interpretable cluster analysis. The "cluster on embedding" strategy is a powerful technique that practitioners can adopt to ensure alignment between visual and analytical results.

- A Ready-to-Use Tool: The universal Python script is a practical tool that can be immediately applied to new datasets, lowering the barrier to entry for performing any rigorous, multi-metric analysis.

6. Conclusion

6.1. Summary of Findings

This project successfully developed and validated a universal pipeline for interpretable cluster analysis. The key findings were:
Clustering on a low-dimensional embedding is a demonstrably superior strategy to clustering on high-dimensional data for producing coherent and visually-aligned clusters.

A multi-faceted interpretation module that combines dual prototypes, decision trees, and SHAP analysis can produce clear, high-fidelity, human-readable explanations for "black box" clusters.

The developed pipeline is robust and universal, proven to be effective across five diverse, high-dimensional datasets from different domains.

6.2. Conclusion

The research presented in this report has successfully met the primary aim set out in the introduction: to develop, implement, and evaluate a robust pipeline for performing interpretable cluster analysis on high-dimensional data.

A comprehensive literature review was conducted, which informed the design of a configurable Python pipeline. This pipeline was implemented and then rigorously evaluated, leading to the selection of a data-driven methodology that balances cluster quality with high interpretability. The primary conclusion of this work is that the sequential process of dimensionality reduction, clustering on the embedding, and post-hoc multi-faceted interpretation represents a powerful and effective paradigm for extracting meaningful insights from complex, high-dimensional data, successfully addressing the "black box" problem in unsupervised learning.

7. Future Work

This project provides a solid foundation that could be extended in several promising directions:

- Formal Python Package: The most logical next step is to formally package the universal script and its dependencies for distribution on the Python Package Index (PyPI). This would transform the project from a script into a reusable library, following the example of packages like clust-learn (Alvarez-Garcia, Ibar-Alonso and Arenas-Parra, 2024).

- Advanced k Selection: Future work could involve integrating more sophisticated methods for determining the optimal number of clusters, such as using the silhouette score or consensus clustering, to help uncover more granular subgroups.

- Exploration of Joint Optimization Models: A more advanced research direction would be to explore models that jointly optimize for dimensionality reduction and clustering simultaneously, such as autoencoders with an integrated clustering loss function.

References

Aggarwal, C.C. (2016) Recommender Systems: The Textbook. Cham: Springer.

Alvarez-Garcia, M., Ibar-Alonso, R. & Arenas-Parra, M. (2024) ‘A comprehensive framework for explainable cluster analysis’, Information Sciences, 663, 120282

Cheriet, M., Arnout, L. and Torres, B. (2020) 'Considerably improving clustering algorithms using UMAP dimensionality-reduction technique: a comparative study', Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW 2020), pp. 37–44.

Kiselev, V.Y., Andrews, T.S. and Hemberg, M. (2019) 'Challenges in unsupervised clustering of single-cell RNA-seq data', Nature Reviews Genetics, 20(5), pp. 273–282.

Moshkovitz, D., Shen, S. and Jain, P. (2020) 'Explainable k-means and k-medians clustering', in Proceedings of the 37th International Conference on Machine Learning (ICML 2020), pp. 6134–6144.

Said, A. (2021) 'A survey on network telemetry: From measurement and collection to analysis and applications', ACM Computing Surveys, 54(6), pp. 1–36.

Xia, P., Singh, G. and Müller, K. (2022) 'Revisiting dimensionality-reduction techniques for visual cluster analysis: an empirical study', IEEE Transactions on Visualization and Computer Graphics, 28(1), pp. 442–452.

Bibliography

Lawless, C. & Günlük, O., 2023. Cluster explanation via polyhedral descriptions. In Proceedings of the 40th International Conference on Machine Learning (ICML 2023), pp. 4567–4575.

Salmanian, Y., Peters, J. & Hsu, C., 2024. DimVis: interpreting visual clusters in dimensionality reduction with explainable boosting machines. Computer Graphics Forum (EuroVis 2024), in the press.

Appendix A: Final Pipeline Source Code

```
#
================================================================================
====
# 1. IMPORTS
#
================================================================================
====
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import time
import json
import warnings

# Preprocessing & Models
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.manifold import TSNE
from sklearn.cluster import KMeans
from sklearn.tree import DecisionTreeClassifier, export_text
import umap

# Metrics & Interpretation
from sklearn.metrics import accuracy_score, silhouette_score,
adjusted_rand_score
from sklearn.manifold import trustworthiness
import shap

warnings.filterwarnings('ignore')

#
================================================================================
====
# 2. CONFIGURATION - MODIFY THIS SECTION FOR A NEW DATASET
#
================================================================================
====


CONFIG = {
    "file_path": "./pisa_spain_sample_v2.csv", #
```

```
    "target_column": None, #
    "id_column": None, #
    "features_to_drop": [] #
}
#
================================================================================
====
# To Use different dataset copy and paste the config from below
#
================================================================================
====

# CONFIG = {
#     "file_path": "./titanic.csv",
#     "target_column": "Survived",
#     "id_column": "PassengerId",
#     "features_to_drop": ["Name", "Ticket", "Cabin"]
# }


# import zipfile
# import os

# zip_file_path = './covtype.csv.zip'
# extracted_dir = '/content/'

# with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:
#     zip_ref.extractall(extracted_dir)

# print(f"File extracted to: {extracted_dir}")
# CONFIG = {
#     "file_path": "/content/covtype.csv",
#     "target_column": "Cover_Type",
#     "id_column": None,
#     "features_to_drop": []
# }



# CONFIG = {
#     "file_path": None,
#     "dataset_name": "mnist_784",
#     "target_column": "class",
#     "id_column": None,
#     "features_to_drop": []
```

```
# }
# from sklearn.datasets import fetch_openml
# mnist = fetch_openml('mnist_784', version=1)
# data, target = mnist.data, mnist.target

# # Use the already loaded data and target variables
# df = data.copy()
# df[CONFIG['target_column']] = target

#
================================================================================
====
# 3. HELPER FUNCTIONS
#
================================================================================
====
def find_optimal_k(data, max_k=10):
    """
    Finds the optimal number of clusters (k) for k-means using the Elbow
Method.

    Args:
        data: The data to be clustered (e.g., the 2D embedding).
        max_k: The maximum number of clusters to test.

    Returns:
        The optimal k value.
    """
    print("\n[INFO] Finding optimal k using the Elbow Method...")
    inertias = []
    k_range = range(1, max_k + 1)

    for k in k_range:
        kmeans = KMeans(n_clusters=k, random_state=42, n_init='auto')
        kmeans.fit(data)
        inertias.append(kmeans.inertia_)

    # Plot the elbow curve
    plt.figure(figsize=(8, 5))
    plt.plot(k_range, inertias, 'bo-')
    plt.xlabel('Number of clusters (k)')
    plt.ylabel('Inertia (WCSS)')
    plt.title('Elbow Method For Optimal k')
    plt.show()
```

```
    p1 = np.array([1, inertias[0]])
    p2 = np.array([max_k, inertias[-1]])

    distances = []
    for i in range(len(inertias)):
        p3 = np.array([i + 1, inertias[i]])
        distance = np.linalg.norm(np.cross(p2-p1, p1-
p3))/np.linalg.norm(p2-p1)
        distances.append(distance)

    optimal_k = distances.index(max(distances)) + 1
    print(f"--> Optimal k identified as: {optimal_k}")
    return optimal_k

def calculate_kmeans_stability(data, k, n_iterations=20):
    """Calculates cluster stability for k-means using bootstrap
resampling."""
    print(f"\n[INFO] Calculating cluster stability with {n_iterations}
iterations...")
    all_labels = []
    for i in range(n_iterations):
        sample_indices = np.random.choice(data.shape[0], data.shape[0],
replace=True)
        bootstrap_data = data[sample_indices]
        clusterer = KMeans(n_clusters=k, random_state=i, n_init='auto')
        clusterer.fit(bootstrap_data)
        full_data_labels = clusterer.predict(data)
        all_labels.append(full_data_labels)

    ari_scores = []
    for i in range(len(all_labels)):
        for j in range(i + 1, len(all_labels)):
            ari_scores.append(adjusted_rand_score(all_labels[i],
all_labels[j]))

    return np.mean(ari_scores) if ari_scores else 0.0

def extract_medoid_prototypes(X_original_df, X_processed, labels):
    """
    Finds the most representative actual data point (medoid) for each
cluster.

    Args:
        X_original_df: The original DataFrame before scaling/encoding.
        X_processed: The scaled/encoded data used for clustering.
```

```
        labels: The cluster labels for each data point.

    Returns:
        A dictionary mapping each cluster label to its prototype's data.
    """
    prototypes = {}
    unique_labels = np.unique(labels)
    for label in unique_labels:
        if label == -1: continue # Skip noise if present

        # Get the indices of points in the current cluster
        idx_in_cluster = np.where(labels == label)[0]

        # Get the processed data points for this cluster
        cluster_points_processed = X_processed[idx_in_cluster]

        # Calculate the centroid of the processed points
        centroid = cluster_points_processed.mean(axis=0)

        # Find the point in the cluster that is closest to the centroid
        distances = np.linalg.norm(cluster_points_processed - centroid,
axis=1)
        medoid_cluster_idx = np.argmin(distances)

        # Get the original index of this prototype point in the full
dataset
        prototype_original_idx = idx_in_cluster[medoid_cluster_idx]

        # Store the original, un-processed data for this prototype
        prototypes[label] = X_original_df.iloc[prototype_original_idx]

    return prototypes

#
================================================================================
====
# 4. MAIN SCRIPT
#
================================================================================
====
if __name__ == '__main__':
    # --- Step 1: Data Loading and Feature Identification ---
    print(f"[INFO] Using dataset: {CONFIG['file_path']}")

    # Load the data from the specified file path
```

```
    try:
        df = pd.read_csv(CONFIG['file_path'])
    except FileNotFoundError:
        print(f"[ERROR] File not found at {CONFIG['file_path']}")
        exit() # Exit if the file is not found
    except Exception as e:
        print(f"[ERROR] An error occurred while loading the file: {e}")
        exit()


    features_to_exclude = CONFIG['features_to_drop'] +
[CONFIG['id_column'], CONFIG['target_column']]
    features_to_exclude = [f for f in features_to_exclude if f is not None
and f in df.columns]
    potential_features_df = df.drop(columns=features_to_exclude)

    numerical_features =
potential_features_df.select_dtypes(include=np.number).columns.tolist()
    categorical_features =
potential_features_df.select_dtypes(include=['object',
'category']).columns.tolist()

    print(f"--> Identified {len(numerical_features)} numerical features.")
    print(f"--> Identified {len(categorical_features)} categorical
features.")

    # --- Step 2: Preprocessing ---
    print("[INFO] Building and applying preprocessing pipeline...")
    numerical_transformer = Pipeline(steps=[('imputer',
SimpleImputer(strategy='median')), ('scaler', StandardScaler())])
    categorical_transformer = Pipeline(steps=[('imputer',
SimpleImputer(strategy='most_frequent')), ('onehot',
OneHotEncoder(handle_unknown='ignore'))])

    preprocessor = ColumnTransformer(
        transformers=[('num', numerical_transformer, numerical_features),
('cat', categorical_transformer, categorical_features)],
        remainder='drop'
    )
    X_processed = preprocessor.fit_transform(potential_features_df)
    feature_names = preprocessor.get_feature_names_out()
    print(f"Data processed successfully. Shape: {X_processed.shape}")

    # --- Step 3: Dimensionality Reduction ---
    print("\n[INFO] Applying dimensionality reduction...")
```

```
    if X_processed.shape[0] < 10000:
        print("[INFO] Using t-SNE for dimensionality reduction...")
        reducer = TSNE(n_components=2, perplexity=30, random_state=42,
n_jobs=-1)
        embedding = reducer.fit_transform(X_processed)
        reduction_method = "t-SNE"
    else:
        print("[INFO] Using UMAP for dimensionality reduction...")
        reducer = umap.UMAP(n_components=2, random_state=42)
        embedding = reducer.fit_transform(X_processed)
        reduction_method = "UMAP"


    # --- Step 4: Clustering with k-means on Embedded Data ---
    print(f"[INFO] Applying k-means clustering on the 2D embedded data
from {reduction_method}...")
    k = 4
    kmeans_clusterer = KMeans(n_clusters=k, random_state=42,
n_init='auto')
    cluster_labels = kmeans_clusterer.fit_predict(embedding)
    print(f"--> Found {k} clusters.")

    # --- Step 5: Calculate All Metrics ---
    print("\n[INFO] Calculating performance metrics...")
    # Trustworthiness is specific to t-SNE, skip if using UMAP
    trust_score = trustworthiness(X_processed, embedding, n_neighbors=15)
if reduction_method == "t-SNE" else None
    sil_score = silhouette_score(embedding, cluster_labels)
    stability_score = calculate_kmeans_stability(embedding, k=k)

    if trust_score is not None:
        print(f"--> Trustworthiness of {reduction_method}:
{trust_score:.4f}")
    print(f"--> Silhouette Score of k-means: {sil_score:.4f}")
    print(f"--> Cluster Stability (Adjusted Rand Index):
{stability_score:.4f}")

    # --- Step 6: Visualization ---
    print("\n[INFO] Generating visualization...")
    df['cluster'] = cluster_labels
    plt.figure(figsize=(10, 7))
    sns.scatterplot(x=embedding[:, 0], y=embedding[:, 1],
hue=df['cluster'], palette="viridis", s=50, alpha=0.8)
```

```
    title = f'{reduction_method} + k-means (Clustered on
Embedding)\nSilhouette: {sil_score:.2f} | Stability:
{stability_score:.2f}'
    if trust_score is not None:
        title += f' | Trust: {trust_score:.2f}'
    plt.title(title, fontsize=16)
    plt.xlabel(f"{reduction_method} Dimension 1");
plt.ylabel(f"{reduction_method} Dimension 2");
plt.legend(title='Cluster'); plt.show()

    # --- Step 7: Interpretation ---
    print("\n[INFO] Generating interpretations...")

    # A. Cluster Prototypes (Centroid - The Average Profile)
    print("\n--- Interpretation Method 1: Cluster Average Profile
(Centroid) ---")
    prototype_cols = numerical_features
    if CONFIG['target_column'] and CONFIG['target_column'] in df.columns:
        prototype_cols = numerical_features + [CONFIG['target_column']]
    centroid_prototypes = df.groupby('cluster')[prototype_cols].mean()
    print(centroid_prototypes.round(2))

    # B. Cluster Prototypes (Medoid - The Best Real Example)
    print("\n--- Interpretation Method 2: Most Representative Example
(Medoid) ---")
    medoid_prototypes = extract_medoid_prototypes(df, X_processed,
cluster_labels)
    for label, data in medoid_prototypes.items():
        print(f"\nCluster {label} Prototype (original data row
{data.name}):")
        print(data)

    # C. Decision Tree Surrogate Model
    print("\n--- Interpretation Method 3: Decision Tree Surrogate ---")
    surrogate_model = DecisionTreeClassifier(max_depth=4, random_state=42)
    surrogate_model.fit(X_processed, cluster_labels)
    surrogate_accuracy = accuracy_score(cluster_labels,
surrogate_model.predict(X_processed))
    print(f"--> Surrogate model accuracy: {surrogate_accuracy:.2%}")
    tree_rules = export_text(surrogate_model,
feature_names=list(feature_names))
    print(tree_rules)

    # D. SHAP Explanations
    print("\n--- Interpretation Method 4: SHAP Summary Plot ---")
```

```
    explainer = shap.TreeExplainer(surrogate_model)
    shap_values = explainer.shap_values(X_processed)
    X_processed_df = pd.DataFrame(X_processed, columns=feature_names)
    shap.summary_plot(shap_values, X_processed_df, plot_type="bar",
class_names=[f"Cluster {i}" for i in range(k)])

    # --- Step 8: Logging Results ---
    # print("\n[INFO] Saving results...")
    # df.to_csv('final.csv', index=False)
```